

Spike: 4**Title:** Tactical Analysis with PlanetWars**Author:** Pattarapol Tantechasa, 103883220**Goals / deliverables:**

- At least two distinct, functional "bot" agents for the PlanetWars simulation (one simple/naïve, one tactical).
- A comprehensive Spike Report saved cleanly as a PDF document.
- The Spike Report must include a clear written explanation of the tactical analysis used, alongside supporting code snippets.
- A numerical comparison of both bots' performance presented via a table (charts are also highly encouraged), aggregated over multiple maps and runs to prove statistical significance.

Technologies, Tools, and Resources used:

- Claude: brainstorm ideas, explain code and debugs.
- Visual Studio Code IDE
- Conda: Python environment manager

Tasks undertaken:

1. Download and Install **Visual Studio Code**
2. Install **Python** and **Pyglet** on your Machine
3. Open the main project downloaded
4. Open terminal and run cd command : **cd Assignments-Labs/Assignment-4/Task-10-Spike-Tactical-Analysis-with-PlanetWars/new-planetwars**
5. Run: **python main.py -p TacticalBot NaiveBot -m map001 -gui**
6. Run Batch: **python batch_runner.py** or **python asymmetric_batch.py**
 - a. Look at console to see game result.
7. Run FogBot: **python main.py -p FogBot NaiveBot -m asym001 -gui**

Design Phase: Tactical Analysis with PlanetWars

Overview

Two bots compete on a shared map. Each owns planets that produce ships every tick based on growth rate. A planet is captured when an attacking fleet outnumbers its defenders. The goal is to eliminate the opponent by capturing all their planets.

Baseline Agent – NaiveBot

Attacks a random target from every owned planet each tick, sending half its ships. No reasoning applied, purely random target selection with a minimum ship threshold of 10.

```
for src in my_planets:
    if src.ships < 10:
        continue
    dest = random.choice(targets)
    gameinfo.planet_order(src, dest, src.ships // 2)
```

Tactical Agent – TacticalBot

Applies three rules each tick per owned planet:

- **Garrison** - keep at least 15 ships on every planet (never leave it defenceless)
- **Affordability** - only attack planets where available ships exceed the target's count + 5
- **Scoring** - among affordable targets, pick the highest scorer:

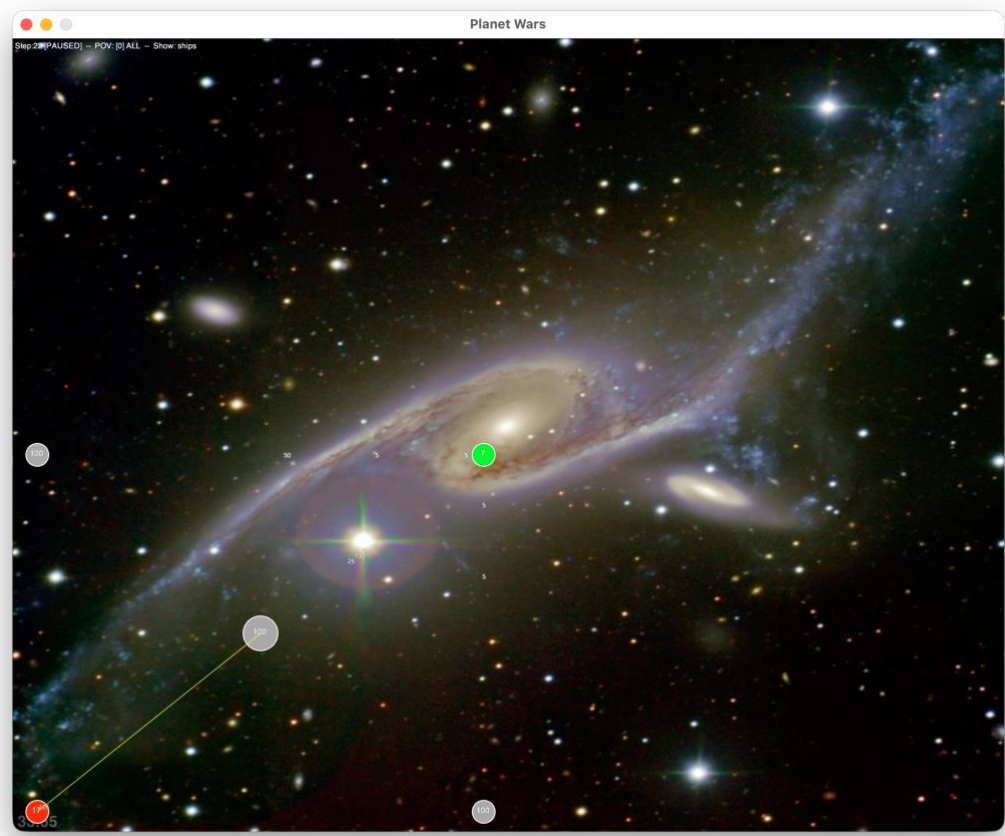
```
score = 40 x growth - 1 x distance
```

Higher growth = more ships per tick. Lower distance = faster arrival. Sends only the minimum ships needed to win.

```
def score(target):
    dist = src.distance_to(target, sqrt=True)
    return GROWTH_WEIGHT * target.growth - DISTANCE_WEIGHT *
dist
best = max(affordable, key=score)
gameinfo.planet_order(src, best, best.ships + ATTACK_BUFFER)
```

Visual Enhancement

Running with `--gui` draws yellow lines from each of TacticalBot's planets to its current target, making the bot's decisions visible in real time.



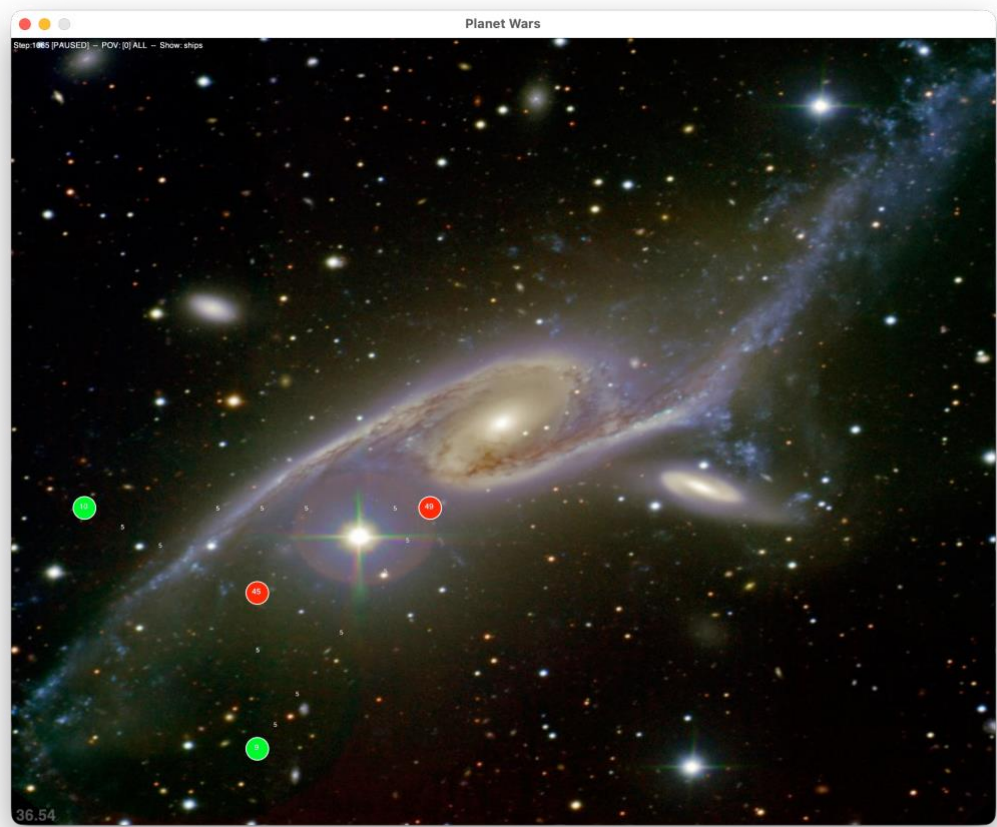
TacticalBot’s current target, yellow line

Results

100 games across 10 maps (10 runs per map).

Map	TacticalBot Wins	NaiveBot Wins	TacticalBot Win%
map001–003, 005–010	90	0	100%
map004	7	3	70%
TOTAL	97	3	97%

TacticalBot won 97/100 games. **NaiveBot’s** 3 wins were all on map004, a map whose layout likely favoured random aggression at that spawn position.



Map004 Screenshots

TacticalBot vs NaiveBot

Aspect	NaiveBot	TacticalBot
Target	Random	Best score (growth + proximity)
Ships sent	Half	Minimum to win + 5
Garrison	None	Always ≥ 15 kept behind
Win rate	3%	97%

Conclusion

Even simple tactical criteria like growth rate, proximity, and garrison discipline produce a dominant advantage over random play. TacticalBot won 97% of games by attacking only when it could afford to win and always defending what it already owned. The exception on map004 shows map geometry can occasionally favour randomness, but the overall data confirms tactical reasoning consistently outperforms naive action selection.

Extension: Asymmetric Maps

Map Design

asym001 is a custom map that deliberately gives each player a different starting situation:

	Player 1 (left side)	Player 2 (right side)
Home planet growth	3	3
Nearby planets	3 planets (growth 4, 5, 5)	3 planets (growth 1, 1, 1)
Total nearby growth	14	5

Player 1 can rapidly snowball by capturing high-growth neighbors, gaining a large ship production lead early. Player 2 is surrounded by weak planets and cannot match that output without fighting toward the centre first.

Results

20 runs per condition (40 total on the asymmetric map).

Condition	Runs	TacticalBot Wins	NaiveBot Wins	TacticalBot Win%
Symmetric baseline (map001–010)	100	97	3	97%
Asym — TacticalBot as P1 (advantaged side)	20	20	0	100%
Asym — TacticalBot as P2 (disadvantaged side)	20	20	0	100%

Analysis

TacticalBot won every game on the asymmetric map, even when placed on the weaker P2 side. The positional disadvantage produced no measurable drag on its win rate. The reason is that NaiveBot cannot exploit an advantaged position. Its random target selection wastes the high-growth neighbors it starts next to, squandering the map's gift. TacticalBot, by contrast, immediately identifies and captures the highest value nearby planets regardless of which side it starts on. Its scoring function ($40 \times \text{growth} - 1 \times \text{distance}$) naturally gravitates toward the high-growth cluster whether that cluster belongs to its side or not.

This result highlights two things:

- **Map asymmetry creates real bias** - the starting positions are genuinely unequal in production potential.
- **Tactical intelligence compensates for positional disadvantage** - a bot that reasons about growth rate and proximity can overcome unfair map conditions that a random bot cannot exploit.

Extension: Fog of War

How Fog of War Works

The game engine already implements fog of war. Every tick, each bot's view of the map is filtered by vision range, only planets and fleets within range of its own planets and fleets are "seen". Every entity carries a `vision_age` counter:

- `vision_age == 0` — seen this tick (fresh, accurate)
- `vision_age > 0` — last seen N ticks ago (stale, ship count is outdated)

TacticalBot ignores `vision_age` entirely. It uses `target.ships` even if the data is 30 ticks old. This is a critical flaw: a planet that had 20 ships when last observed may now have 80.

FogBot Strategy

FogBot extends TacticalBot with two fog-aware behaviors:

1. Stale-data compensation

Before deciding whether to attack and how many ships to send, FogBot adjusts its estimate of the target's current ship count:

```
estimated_ships = target.ships + target.growth * target.vision_age
```

A planet with 20 last-known ships, growth rate 3, and `vision_age` of 20 is estimated to now have 80 ships. FogBot uses this value for affordability checks and to decide how many ships to send which preventing failed attacks from under-estimation.

2. Scouting

Once per tick, FogBot finds the highest-priority unseen planet (scored by `growth * vision_age`) and sends a tiny 3-ship scout fleet toward it:

```
scout_target = max(stale_targets, key=lambda p: p.growth *  
p.vision_age)  
gameinfo.planet_order(scout_src, scout_target, SCOUT_SHIPS)
```

As the scout travels it reveals planets along its path, refreshing the bot's intelligence on key high-growth targets before committing a full attack fleet.

Outcome

FogBot runs without errors under the engine's built-in fog-of-war facade. Its stale-data compensation prevents the most common fog-of-war failure mode (sending too few ships to capture a grown planet), while scouting ensures high-value enemy planets don't stay dark for long.